

Answer ALL questions.

Write your answers clearly and legibly. Marks are awarded for correct logic, syntax, and structure.

No electronic devices are permitted.

This is a closed-book examination.

(100 marks)

-
1. Create a class called **NetworkDevice** with appropriate instance variables to store a device **ID**, **IP address**, **device name**, whether the device **is_online**, and its **threat_level** as a numerical value.
 - a. Write a **parameterized constructor** for the NetworkDevice class that initializes all the instance variables.
 - b. Write a method called **displayStatus()** inside the NetworkDevice class that prints all device details to the console in a readable format.
 - c. Write a method called **isCritical()** that checks the device's threat level and returns true if the threat level is 7 or above, and false otherwise.
 - d. Call this method in **main()** and print the result. In the **main() method**, create a **NetworkDevice object**, call **displayStatus()** and **isCritical()**, and print the result. Write the expected output of your program in the space provided.

 2. Create a class called **UserAccount**. Declare instance variables for a **username**, **email address**, **account status**, **failed login attempts**, and a **password**. Apply the correct access modifier to each variable to follow the principles of encapsulation.
 - a. Write **getter and setter methods** for the username and failed login attempts. The setter for failed login attempts must reject any negative values and print an appropriate error message if an invalid value is passed.
 - b. Write a method called **login()** that accepts a password string and compares it to the stored password. If the passwords match, print a success message and reset failed login attempts to zero. If not, increment the failed login attempts and print a failure message.
 - c. Write a method called **lockCheck()** that checks the number of failed login attempts. If the attempts reach 3 or more, set the account status to locked and print a warning message. Otherwise print that the account is active.
 - d. In **main()**, create a UserAccount object with suitable values. Demonstrate the following:
 - a. Print the username using the getter.
 - b. Try setting a negative value for failed login attempts.
 - c. Attempt a correct login and call lockCheck().
 - d. Attempt three incorrect logins and call lockCheck() again.

3. Create a class called **ThreatLog** with five individual integer variables to store threat severity scores **score1, score2, score3, score4, score5**.
 - a. Write a **parameterized constructor** that accepts all five scores and assigns them.
 - b. Write a method called **printReport()** inside ThreatLog that prints each score along with its threat classification using the following levels **80 and above: Critical, 60 to 79: High, 40 to 59: Medium, below 40: Low**.
 - c. Write a method called **getHighestThreat()** that compares all five scores using if statements and returns the highest one.
 - d. Write a method called **countCritical()** that counts how many scores are 80 or above and returns the count.
 - e. In **main()**, create a ThreatLog object with the following scores: 91, 55, 38, 74, 62. Call all three methods and write the expected output.

4. A cybersecurity firm wants to track its security analysts. Create a class called **SecurityAnalyst** that contains the following:
 - a. Appropriate **instance variables** to store an **analyst ID, name, specialization, and clearance level**.
 - b. A **default constructor** that assigns default values to all instance variables and increments totalAnalysts.
 - c. A **parameterized constructor** that accepts values for all instance variables and increments totalAnalysts.
 - d. A method called **getProfile()** that prints all analyst details.
 - e. A **static method** called **getTotalAnalysts()** that returns the value of totalAnalysts.
 - f. In **main()**, create one analyst using the default constructor and one using the parameterized constructor. Call **getProfile()** on both objects, then call **getTotalAnalysts()** and print the result. Write the expected output of your program.